Encapsulation in Java – Final Revision

Quick Revision:

- Encapsulation means wrapping data + nd methods
- Data hiding is part of encapsulation
- Achieved using private variables
- Access provided using public methods

Important Keywords (boxed neatly):

encapsulation	private	public	getter
setter	data hiding	Java Bean	

MCQ / Viva Important Points:

- Encapsulation improves security
- private keyword is core of encapsulation
- Getter reads data, Setter modifies data
- Validation logic can be added in setter
- Java Bean class follows encapsulation rules

True / False Style:

- Encapsulation supports data hiding (True)
- Variables should be public (False)
- Getter can modify data (False)
- Setter can validate data (True)

One-line Exam Answers:

- Encapsulation binds data and methods
- Used to protect data from misuse

Encapsulation makes Java programs secure and maintainab

Encapsulation - Summary & Key Points

Core Concept:

Encapsulation is the technique of binding data and methods into a single unit.

How Encapsulation is Achieved:

- Declare variables as private variables
- Provide public getter and setter methods

Benefits of Encapsulation:

- Data hiding
- Security
- Controlled access
- Easy maintenance
- Flexibility

Write definition first in exams

Important Keywords:

- private
- public
- getter
- setter
- data hiding
- Java Bean

Use keywor for better marks

Common Mistakes:

- Making variables public
- Accessing data directly
- Skipping validation in setter

Encapsulation protects data and improves code quality

Encapsulation - Interview & Exam Questions

Very Short Questions:

Q1. What is Encapsulation?
- Wrapping data and methods into a single unit. Data + Methods

Q2. Which keyword is mainly used?
- private

Q3. How is encapsulation achieved in Java?
- Using private variables
- Using public getter & setter

Write definiti clearly in exams

Use keywor for full marks

Short Answer Questions:

Q4. Difference between Encapsulation and Data Hiding?
- Encapsulation is broader concept
- Data hiding is part of encapsulation

Q5. Why variables are kept private?
- To protect data
- To control access

One-line Exam Points:

- Encapsulation improves security
- Prevents direct access to data
- Supports maintainable code

Encapsulation questions are easy scoring

Read-Only & Write-Only Classes

Read-Only Class:

A class in which data can be read but cannot be modified.

How to achieve:

- Make variable private — Only getter
- Provide only getter method — No setter

```
class Student {
    private int roll = 101;

    public int getRoll() {
        return roll;
    }
}
```

- roll value can be accessed
- roll value cannot be changed

Read-only and write-only improve security

Used in re application

Write-Only Class:

A class in which data can be modified but cannot be read.

How to achieve:

- Make variable private — Only setter
- Provide only setter method — No getter

```
class Account {
    private int balance;

    public void setBalance(int b) {
        balance = b;
    }
}
```

- balance can be modified
- balance value cannot be accessed directly

Read-only and write only impro security

Used in re application

Getter and Setter control data accessibility

Java Bean Class & Encapsulation

Java Bean Class:

A Java Bean is a special class that follows encapsulation rules. Private variables and Public getter and setter.

Rules of Java Bean:

- All variables must be private
- Public getter and setter methods
- Must have public no-argument constructor
- Implements Serializable (optional)
- Encapsulation standard

```
class Student implements Serializable {
    private int roll;
    private String name;

    public Student() { }

    public int getRoll() {
        return roll;
    }
    public void setRoll(int r) {
        roll = r;
    }
    public String getName() {
        return name;
    }
    public void setName(String n) {
        name = n;
    }
}
```

- Data is hidden using private
- Access controlled by methods

Used in JSP, Spring, Hiberna

Very importa for interviev

Java Bean is a perfect example of encapsulation

Encapsulation with Validation

Validation in Encapsulation:

Encapsulation allows data validation before assignment. Controlled access

Why validation is needed:

- Prevent invalid data
- Improve security
- Maintain Data integrity

Business log
inside sette

Very importa
interviews cond

```
class User {
    private int age;

    public void setAge(int a) {
        if (a > 0 && a < 120) {
            age = a;
        }
    }

    public int getAge() {
        return age;
    }
}
```

- age is private
- Only valid values are allowed
- Invalid data is ignored

Encapsulation allows validation and safe data handlin

Encapsulation with Real-Life Example

Real-Life Example:

Encapsulation is similar to a capsule or ATM machine.

Example 1: Capsule

- Medicine is inside capsule
- User cannot see internal contents
- Only effect is visible

Internal details hidden

Example 2: ATM Machine

- User inserts card
- Enters PIN
- Performs transaction
- Internal processing is hidden

User sees only interface

Java Connection:

- Variables are private
- Methods act as interface
- Internal logic is hidden

Very easy to remember in exams

Real-life analogy helps understanding

```
class BankAccount {
    private int balance = 1000;
    public void withdraw(int amt) {
        balance = balance - amt;
    }
    public int getBalance() {
        return balance;
    }
}
```

- balance is hidden
- Access allowed via methods only

Encapsulation hides internal details from user

Encapsulation vs Data Hiding

Encapsulation:

- Wrapping data and methods together
- Complete OOP concept – Data + Methods
- Achieved using class

Data Hiding:

- Protecting data from outside access
- Part of encapsulation – Data protection
- Achieved using private keyword

Feature	Encapsulation	Data Hiding
Definition	Wrapping data	Hiding data
Achieved using	Class	private keyword
Scope	Broad concept	Limited concept
Purpose	Organization	Security

```
class Demo {
   private int x;   // data hiding

   public void show() {
      // encapsulation
   }
}
```

- Encapsulation includes data hiding
- Data hiding focuses on security

Encapsulati
is broade

Common
exam
compariso

Encapsulation includes data hiding

Advantages of Encapsulation

Advantages of Encapsulation:

1. Data Hiding - Data protection
   - Internal data is protected
   - Direct access is not allowed

Very importan
exam questio

2. Better Control - Controlled access
   - Validation can be applied
   - Only safe values allowed

3. Improved Security
   - Prevents misuse of data
   - Reduces errors

Encapsulatio
improves
code qualit

4. Flexibility - Flexible design
   - Internal code can change
   - Without affecting other classes

5. Improved Maintainability
   - Code is easy to manage
   - Changes are localized

```
public void setAge(int a) {
    if (a > 0) {
        age = a;
    }
}
```

- Validation is possible using setters
- Data remains safe

Encapsulation makes code secure and maintainable

Getter and Setter Methods

Getter Method:

A method used to Read data the value of a private variable. This is the get method

Setter Method:

A method used to Modify the value of a private variable. This is the set method

Naming Convention:

- getVariableName()
- setVariableName(value)

Follows JavaBeans standard

Used in framework

```
class Student {
    private int roll;

    public void setRoll(int r) {
        roll = r;
    }

    public int getRoll() {
        return roll;
    }
}
```

- Setter assigns valu
- Getter returns valu

Getter and Setter provide controlled access

Access Modifiers & Data Hiding

Access Modifiers:

Access modifiers control the visibility of variables and methods. Controls visibility

Types of Access Modifiers:

- private
- public
- protected
- default

Data Hiding:

Hiding internal data from outside world is called data hiding. Data protection

private is key for encapsulatio

Data hiding improves security

How Encapsulation achieves Data Hiding:

- Declare variables as private
- Provide public getter and setter methods

```
class Account {
    private double balance;

    public void setBalance(double b) {
        balance = b;
    }
    public double getBalance() {
        return balance;
    }
}
```

- balance cannot be accessed directly
- Controlled access is provided

Encapsulation + private = Data hiding

Encapsulation in Java

Introduction

Encapsulation :

Encapsulation is the process of wrapping data and methods into a single unit.

In Java, encapsulation is achieved by making variables private and accessing them using methods.

Why we use Encapsulation :

- Data hiding is core of encapsulation
- Security
- Better control over data
- Improves maintainability

Real-life example

Capsule → Medicine inside
Class → Data + Methods

```
class Student {
    private int roll;
    public void setRoll(int r) {
        roll = r;
    }
    public int getRoll() {
        return roll;
    }
}
```

- Data is private
- Access through methods

private variables cannot b accessed directly

Uses get and sett methods

Encapsulation protects data from misuse